ISLAMIC UNIVERSITY OF TECHNOLOGY

DHAKA, BANGLADESH

ORGANIZATION OF ISLAMIC COOPERATION

DATABASE MANAGEMENT SYSTEMS LAB

Name: Antara Arifa Mullick

Student ID: 210042132

Department: Computer Science and Engineering

Program: B.Sc. in Software Engineering

Report for OOC-II Project

Library Management System (LMS)

Project Overview:

The Library Management System project is designed to efficiently manage library resources, members, and transactions through a user-friendly C# application. The system aims to provide a comprehensive solution for library members, ensuring smooth operations and effective resource utilization.

Features:

Manage Books:

- New books are added to the library system by providing essential details such as title, author.

Display Library Items:

- The application allows users to view the entire collection of library items, providing detailed information about each book, including its title, author, borrowing status, and reservation status.

Manage Members:

- New library members are added to the library system by providing their names. We have two types of member Borrow Member and Reserve Member as the child of Library Member.

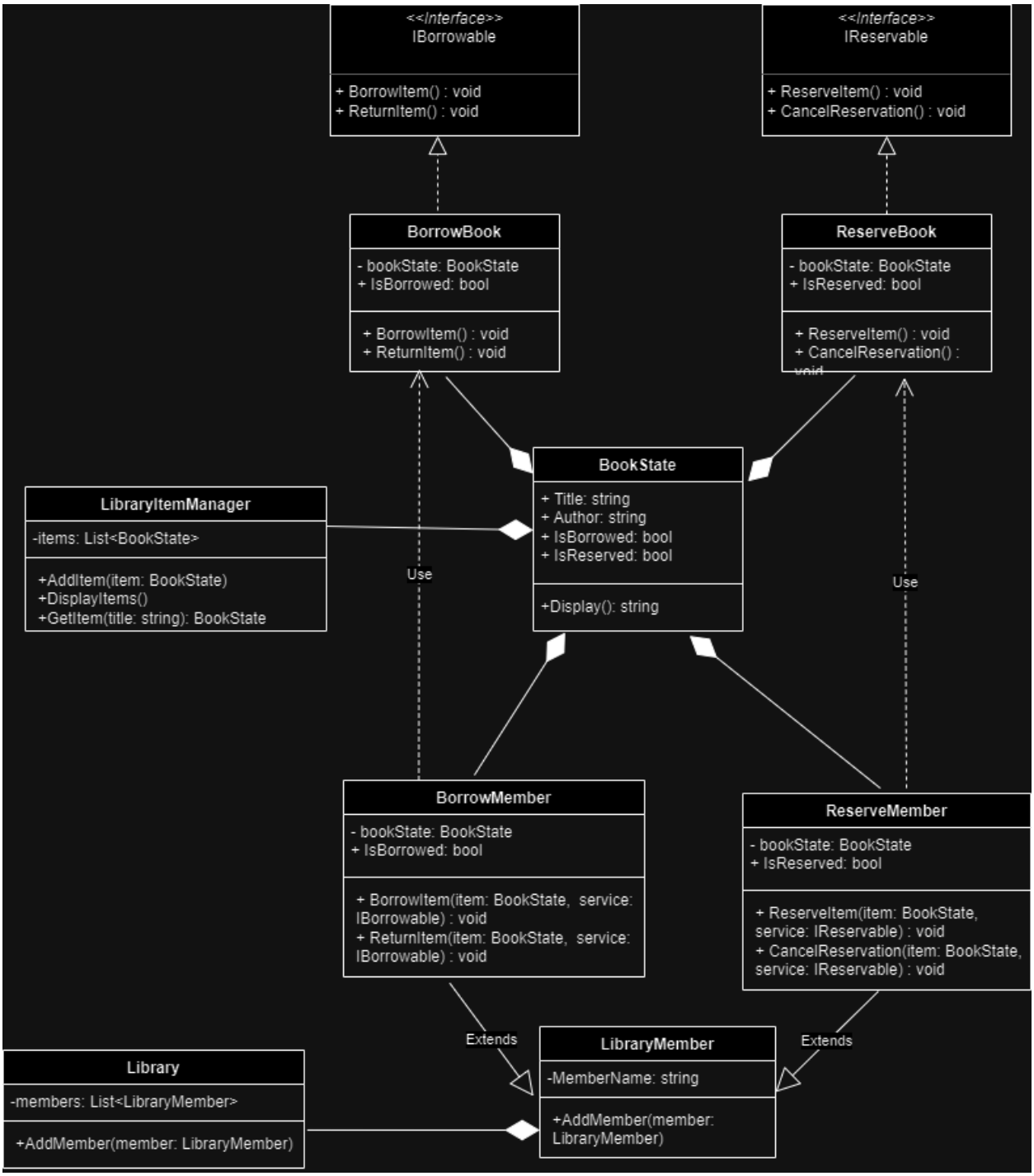
Borrow System:

- Library members can borrow and return books using the application, and the system updates the borrowing status accordingly.

Reservation System:

- Members can reserve and cancel reservation of books using the application, and the system manages reservation status.

UML Diagram:



How the project follows SOLID principles:

Using Single Responsibility Principle:

In this project, the Single Responsibility Principle (SRP) is diligently followed. Each class is designed with a distinct and singular responsibility.

For example, the BookState class manages the state of a book. BorrowBook and ReserveBook handle borrowing and reserving books, respectively . LibraryMember, BorrowMember, and ReserveMember are responsible for managing library members. This adherence to SRP enhances modularity and ensures that changes in one part of the system do not necessitate modifications in unrelated areas.

Using Open Closed Principle:

The Open/Closed Principle (OCP) is evident in the project through the use of abstraction and extension mechanisms. Classes are designed to be extended without modifying their source code.

Interfaces, such as IBorrowable and IReservable, allow for easy extension. So, BorrowBook and ReserveBook can be extended for new borrowing or reserving strategies. This approach aligns with OCP, fostering a modular and extensible design.

Using Liskov Substitution Principle:

The Liskov Substitution Principle (LSP) is well-implemented in the design of the book hierarchy. Derived classes can be used interchangeably with their base class.

BorrowMember and ReserveMember (derived classes) can be treated as LibraryMember(base class) . Also BorrowBook and ReserveBook classes implement the IBorrowable and IReservable interfaces. Instances of these classes can be substituted for any other class that implements the same interfaces This inheritance relationship ensures code flexibility, extensibility, and a consistent inheritence adherence to LSP.

Using Interface Segregation Principle:

Interface Segregation Principle (ISP) is applied in the project to ensure that classes only depend on interfaces relevant to their specific functionalities. They do not force implementing unnecessary methods.

For instance, the IBorrowable represents the ability to borrow and return items and IReservable represents the ability to reserve and cancel reservations for itemseach interface has a clear and specific purpose, and classes can implement only the interfaces relevant to their functionalities. This adherence to ISP enhances code clarity and allows for the creation of new interfaces without imposing irrelevant methods on implementing classes.

Using Dependency Injection Principle:

The Dependency Inversion Principle (DIP) is incorporated in the project by relying on abstractions such as interfaces. High-level modules are not directly dependent on low-level modules.

LibraryItemManager and Library depends on abstractions such as BookState,not on concrete implementations. BorrowBook and ReserveBook depend on interfaces IBorrowable and IReservable. So, this aligns with DIP and facilitating extensibility without modifying existing high-level modules.

We can see that the project follows the SOLID principles. Each class has a clear,single responsibility, is open for extension, can be used interchangeably with its base class or interface, interfaces are focused, and high-level modules depend on abstractions. This adherence to SOLID principles contributes to the overall maintainability and flexibility of the library management system.

Code smells that are avoided:

Large Class:

- The classes in the projects are well-organized and focused on specific responsibilities.

Long Method:

-The methods in the projects are concise and focused on implementing only necessary functionality.

Feature Envy:

- Methods appear to be appropriately placed within their respective classes, and there is no evidence of one class excessively accessing the internals of another class.

Duplicated Code:

- The code does not have explicit duplication. Common functionality is encapsulated in methods and interfaces.

Inappropriate Naming:

- Observation: The variable and method names appear consistent and follow a clear naming convention.

Git link of project –

https://github.com/AntaraArifa/Library_Management_System